```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

// Enum types for better type safety
type PartnerType string
type AgentType string
type StatusType string

const (
// Partner types
PartnerTypeInsurance PartnerType = "insurance"
PartnerTypeBank PartnerType = "bank"
PartnerTypeDealer PartnerType = "dealer"
PartnerTypeOther PartnerType = "other"

// Agent types
AgentTypeIndividual AgentType = "individual"
AgentTypeCorporate AgentType = "corporate"

// Status types
StatusActive StatusType = "active"
StatusInactive StatusType = "inactive"
StatusPending StatusType = "pending"
StatusApproved StatusType = "approved"
StatusRejected StatusType = "rejected"
StatusPaid StatusType = "paid"
StatusUnpaid StatusType = "unpaid"
StatusAssigned StatusType = "assigned"
StatusCompleted StatusType = "complete"
StatusInprogress StatusType = "inprogress"
StatusIntervention StatusType = "intervention"
StatusDraft StatusType = "draft"
StatusInvoiced StatusType = "invoiced" // New status for invoiced requests
)

// Customer represents customer data for valuation service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
```

```
Email string `json:"email" gorm:"not null;unique"`
Phone string `json:"phone" gorm:"not null;unique"`
FirstName string `json:"first_name" gorm:"not null"`
LastName string `json:"last_name" gorm:"not null"`
Status string `json:"status" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:CustomerID"`
VehicleInsurances []VehicleInsurances `json:"vehicle_insurances"
gorm:"foreignKey:CustomerID"`
}

// Partners represents partner organizations
type Partners struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
PartnerType PartnerType `json:"partner_type" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
Agents []Agents `json:"agents" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:PartnerID"`
}

// Agents represents partner agents
type Agents struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
AgentType AgentType `json:"agent_type" gorm:"not null"`
PartnerID string `json:"partner_id" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
```

```
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
Partner Partners `json:"partner" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:AgentID"`
}

// ValuationTypes represents different types of valuations
type ValuationTypes struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null;unique"`
Description string `json:"description"`
MemberPrice float64 `json:"member_price"`
NonMemberPrice float64 `json:"non_member_price"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

//ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:ValuationTypeId"`
}

// Branches represents company branches
type Branches struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
//DeletedAt time.Time `json:"deleted_at"`
}

// VehicleInsurances represents vehicle insurance information
type VehicleInsurances struct {
```

```
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InsuranceCompany string `json:"insurance_company" gorm:"not null"`
PolicyNumber string `json:"policy_number" gorm:"unique;not null"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
CustomerID string `json:"customer_id" gorm:"not null"`
ExpiryDate time.Time `json:"expiry_date" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
}

// Valuer represents professional valuers
type Valuer struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Email string `json:"email" gorm:"unique"`
Phone string `json:"phone" gorm:"not null;unique"`
Name string `json:"name"`
Location string `json:"location" gorm:"not null"`
CurrentCoordinate string `json:"current_coords" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:ValuerID"`
}

// ImageTypes represents different types of images
type ImageTypes struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
ImageTypeName string `json:"image_type_name" gorm:"not null;unique"`
TypeDescription string `json:"type_description"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
```

```
ValuationRequestImages []ValuationRequestImages
`json:"valuation_request_images" gorm:"foreignKey:ImageTypeID"`
}

type ValuationRequestImages struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
RequestID string `json:"request_id" gorm:"not null"`
ImageTypeID string `json:"image_type_id" gorm:"not null"`
ImageURL string `json:"image_url" gorm:"not null"`
ImageLatitude *float64 `json:"image_latitude,omitempty"`
ImageLongitude *float64 `json:"image_longitude,omitempty"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships - Make sure these don't cause circular references
ValuationRequest ValuationRequests `json:"valuation_request,omitempty"
gorm:"foreignKey:RequestID"`
ImageType ImageTypes `json:"image_type" gorm:"foreignKey:ImageTypeID"`
}

// ValuationRequests represents valuation requests - UPDATED: Added InvoiceID
// ValuationRequests represents valuation requests - UPDATED: Added InvoiceID
// FIXED: ValuationRequests with corrected Agent relationship
type ValuationRequests struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
PartnerID *string `json:"partner_id,omitempty"`
AgentID *string `json:"agent_id,omitempty"`
CustomerID string `json:"customer_id" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
LocationCoordinates *string `json:"location_coordinates,omitempty"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
ValuerID *string `json:"valuer_id,omitempty"`
ApproverID *string `json:"approver_id,omitempty"`
ValuationTypeId string `json:"valuation_type_id"`
InvoiceID *string `json:"invoice_id,omitempty"`
Status StatusType `json:"status" gorm:"default:pending"`
CalculatedValue *string `json:"calculated_value" gorm:"default:null"`
CrspPrice *string `json:"crsp_price" gorm:"default:null"`
EngineMarks *string `json:"engine_marks" gorm:"default:null"`
AgeMarks *string `json:"age_marks" gorm:"default:null"`
```

```
MileageMarks *string `json:"mileage_marks" gorm:"default:null"`
BodyMarks *string `json:"body_marks" gorm:"default:null"`
FacilitiesMarks *string `json:"facilities_marks" gorm:"default:null"`
MaintenanceMarks *string `json:"maintenance_marks" gorm:"default:null"`
TireMarks *string `json:"tire_marks" gorm:"default:null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// FIXED: Relationships with proper nullable handling
Partner *Partners `json:"partner,omitempty"
gorm:"foreignKey:PartnerID;constraint:OnUpdate:CASCADE,OnDelete:SET NULL"`
Agent *Agents `json:"agent,omitempty"
gorm:"foreignKey:AgentID;constraint:OnUpdate:CASCADE,OnDelete:SET NULL"`
Customer Customer `json:"customer"
gorm:"foreignKey:CustomerID;constraint:OnUpdate:CASCADE,OnDelete:CASCADE"`
ValuationType ValuationTypes `json:"valuation_type"
gorm:"foreignKey:ValuationTypeId;constraint:OnUpdate:CASCADE,OnDelete:CASCAD
E"`
Valuer *Valuer `json:"valuer,omitempty"
gorm:"foreignKey:ValuerID;constraint:OnUpdate:CASCADE,OnDelete:SET NULL"`
Invoice *ValuationInvoices `json:"invoice,omitempty"
gorm:"foreignKey:InvoiceID;constraint:OnUpdate:CASCADE,OnDelete:SET NULL"`
InspectionAnswers []InspectionAnswers `json:"inspection_answers,omitempty"
gorm:"foreignKey:ValuationRequestID"`
ValuationRequestImages []ValuationRequestImages
`json:"valuation_request_images,omitempty" gorm:"foreignKey:RequestID"`
}

// ValuationInvoices represents invoices for valuation services - UPDATED: Removed
ValuationRequestID
type ValuationInvoices struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InvoiceNumber string `json:"invoice_number" gorm:"unique;not null"`
AmountBilled float64 `json:"amount_billed" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:unpaid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// ONLY the requests relationship - no payments relationship to avoid FK conflicts
ValuationRequests []ValuationRequests `json:"valuation_requests,omitempty"
gorm:"foreignKey:InvoiceID"`
}
```

```
// ValuationPayments - FIXED GORM TAGS
type ValuationPayments struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InvoiceNumber string `json:"invoice_number" gorm:"not null;index"` // Added index
for FK performance
ReceiptNumber string `json:"receipt_number" gorm:"unique;not null"`
AmountPaid float64 `json:"amount_paid" gorm:"not null"`
PaymentMethod string `json:"payment_method,omitempty"`
PaymentDate *time.Time `json:"payment_date,omitempty"`
Notes string `json:"notes,omitempty"`
Status StatusType `json:"status" gorm:"default:paid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// OPTION 1: Remove the relationship completely to avoid GORM confusion
// ValuationInvoice ValuationInvoices `json:"valuation_invoice,omitempty"`
}

// Inspection Templates
type InspectionTemplate struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
TemplateName string `json:"template_name" gorm:"not null;unique"`
TemplateDescription string `json:"template_description" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
InspectionSections []InspectionSections `json:"inspection_sections"
gorm:"foreignKey:TemplateID"`
}

// Updated InspectionSections to include TemplateID
type InspectionSections struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
TemplateID string `json:"template_id"` // Added this field
SectionName string `json:"section_name" gorm:"not null"`
Description string `json:"description"`
IsWeighted bool `json:"is_weighted" gorm:"null"`
WeightField string `json:"weight_field" gorm:"null"`
CreatedAt time.Time `json:"created_at"`
```

```
UpdatedAt time.Time `json:"updated_at"`

// Relationships
InspectionTemplate InspectionTemplate `json:"inspection_template"
gorm:"foreignKey:TemplateID"`
InspectionQuestions []InspectionQuestions `json:"inspection_questions"
gorm:"foreignKey:SectionID"`
}

// InspectionQuestions represents questions within each inspection section
type InspectionQuestions struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
SectionID string `json:"section_id" gorm:"not null"`
Question string `json:"question" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
InspectionSection InspectionSections `json:"inspection_section"
gorm:"foreignKey:SectionID"`
InspectionAnswers []InspectionAnswers `json:"inspection_answers"
gorm:"foreignKey:QuestionID"`
}

// InspectionAnswers represents answers to inspection questions for specific
valuation requests
type InspectionAnswers struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
QuestionID string `json:"question_id" gorm:"not null"`
ValuationRequestID string `json:"valuation_request_id" gorm:"not null"`
Answer bool `json:"answer" gorm:"not null"` // true or false
Description string `json:"description,omitempty"` // Optional additional description
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
InspectionQuestion InspectionQuestions `json:"inspection_question"
gorm:"foreignKey:QuestionID"`
ValuationRequest ValuationRequests `json:"valuation_request"
gorm:"foreignKey:ValuationRequestID"`
}
```

```
// BeforeCreate hooks for UUID generation
func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (p *Partners) BeforeCreate(tx *gorm.DB) error {
if p.ID == "" {
p.ID = uuid.New().String()
}
return nil
}

func (a *Agents) BeforeCreate(tx *gorm.DB) error {
if a.ID == "" {
a.ID = uuid.New().String()
}
return nil
}

func (vt *ValuationTypes) BeforeCreate(tx *gorm.DB) error {
if vt.ID == "" {
vt.ID = uuid.New().String()
}
return nil
}

func (b *Branches) BeforeCreate(tx *gorm.DB) error {
if b.ID == "" {
b.ID = uuid.New().String()
}
return nil
}

func (vi *VehicleInsurances) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

func (v *Valuer) BeforeCreate(tx *gorm.DB) error {
```

```
if v.ID == "" {
v.ID = uuid.New().String()
}
return nil
}

func (it *ImageTypes) BeforeCreate(tx *gorm.DB) error {
if it.ID == "" {
it.ID = uuid.New().String()
}
return nil
}

func (vri *ValuationRequestImages) BeforeCreate(tx *gorm.DB) error {
if vri.ID == "" {
vri.ID = uuid.New().String()
}
return nil
}

func (vr *ValuationRequests) BeforeCreate(tx *gorm.DB) error {
if vr.ID == "" {
vr.ID = uuid.New().String()
}
return nil
}

func (vi *ValuationInvoices) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

// BeforeCreate hook for ValuationPayments
func (vp *ValuationPayments) BeforeCreate(tx *gorm.DB) error {
if vp.ID == "" {
vp.ID = uuid.New().String()
}
// Set payment date to current time if not provided
if vp.PaymentDate == nil {
now := time.Now()
vp.PaymentDate = &now
}
```

```
return nil
}

// GetTotalAmountPaid calculates total amount paid for an invoice
func GetTotalAmountPaidForInvoice(db *gorm.DB, invoiceNumber string) (float64,
error) {
var totalPaid float64
err := db.Model(&ValuationPayments{}).
Where("invoice_number = ? AND status = ?", invoiceNumber, StatusPaid).
Select("COALESCE(SUM(amount_paid), 0)").
Scan(&totalPaid).Error
return totalPaid, err
}

// CheckInvoiceFullyPaid checks if an invoice is fully paid
func CheckInvoiceFullyPaid(db *gorm.DB, invoiceNumber string) (bool, error) {
var invoice ValuationInvoices
if err := db.Where("invoice_number = ?", invoiceNumber).First(&invoice).Error; err !=
nil {
return false, err
}

totalPaid, err := GetTotalAmountPaidForInvoice(db, invoiceNumber)
if err != nil {
return false, err
}

return totalPaid >= invoice.AmountBilled, nil
}

// GetOutstandingAmount calculates outstanding amount for an invoice
func GetOutstandingAmountForInvoice(db *gorm.DB, invoiceNumber string)
(float64, error) {
var invoice ValuationInvoices
if err := db.Where("invoice_number = ?", invoiceNumber).First(&invoice).Error; err !=
nil {
return 0, err
}

totalPaid, err := GetTotalAmountPaidForInvoice(db, invoiceNumber)
if err != nil {
return 0, err
}
```

```
outstanding := invoice.AmountBilled - totalPaid
if outstanding < 0 {
outstanding = 0 // Prevent negative outstanding amounts
}

return outstanding, nil
}

func (it *InspectionTemplate) BeforeCreate(tx *gorm.DB) error {
if it.ID == "" {
it.ID = uuid.New().String()
}
return nil
}

func (is *InspectionSections) BeforeCreate(tx *gorm.DB) error {
if is.ID == "" {
is.ID = uuid.New().String()
}
return nil
}

func (iq *InspectionQuestions) BeforeCreate(tx *gorm.DB) error {
if iq.ID == "" {
iq.ID = uuid.New().String()
}
return nil
}

func (ia *InspectionAnswers) BeforeCreate(tx *gorm.DB) error {
if ia.ID == "" {
ia.ID = uuid.New().String()
}
return nil
}


// valuation_invoice_repository.go
package repositories

import (
"time"
```

```
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/models"
"gorm.io/gorm"
)

type ValuationInvoiceRepository struct {
db *gorm.DB
}

func NewValuationInvoiceRepository(db *gorm.DB) *ValuationInvoiceRepository {
return &ValuationInvoiceRepository{db: db}
}

// CreateValuationInvoice creates a new valuation invoice
func (r *ValuationInvoiceRepository) CreateValuationInvoice(valuationInvoice *models.ValuationInvoices) error {
return r.db.Create(valuationInvoice).Error
}

// UPDATED: GetAllValuationInvoices retrieves all valuation invoices with pagination
func (r *ValuationInvoiceRepository) GetAllValuationInvoices(limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// GetValuationInvoicesCount returns total count of valuation invoices
func (r *ValuationInvoiceRepository) GetValuationInvoicesCount() (int64, error) {
var count int64
err := r.db.Model(&models.ValuationInvoices{}).Count(&count).Error
return count, err
}
```

```
// UPDATED: GetValuationInvoiceByID retrieves a valuation invoice by ID
func (r *ValuationInvoiceRepository) GetValuationInvoiceByID(invoiceID string)
(*models.ValuationInvoices, error) {
var valuationInvoice models.ValuationInvoices
err := r.db.Where("id = ?", invoiceID).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
Preload("ValuationRequests.ValuationRequestImages"). // Load images for each
request
Preload("ValuationRequests.ValuationRequestImages.ImageType"). // Load image
types
// REMOVED: Preload("ValuationPayments") - this was causing the error
First(&valuationInvoice).Error
return &valuationInvoice, err
}

// UPDATED: GetValuationInvoiceByInvoiceNumber retrieves a valuation invoice by
invoice number
func (r *ValuationInvoiceRepository)
GetValuationInvoiceByInvoiceNumber(invoiceNumber string)
(*models.ValuationInvoices, error) {
var valuationInvoice models.ValuationInvoices
err := r.db.Where("invoice_number = ?", invoiceNumber).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
Preload("ValuationRequests.ValuationRequestImages"). // Load images for each
request
Preload("ValuationRequests.ValuationRequestImages.ImageType"). // Load image
types
// REMOVED: Preload("ValuationPayments") - this was causing the error
First(&valuationInvoice).Error
return &valuationInvoice, err
}

// UpdateValuationInvoice updates an existing valuation invoice
func (r *ValuationInvoiceRepository) UpdateValuationInvoice(valuationInvoice
*models.ValuationInvoices) error {
return r.db.Save(valuationInvoice).Error
```

```
}

// DeleteValuationInvoice soft deletes a valuation invoice
func (r *ValuationInvoiceRepository) DeleteValuationInvoice(invoiceID string) error {
return r.db.Delete(&models.ValuationInvoices{}, "id = ?", invoiceID).Error
}

// UPDATED: GetValuationInvoicesByStatus retrieves valuation invoices by status
func (r *ValuationInvoiceRepository) GetValuationInvoicesByStatus(status string,
limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Where("status = ?", status).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// REMOVED: GetValuationInvoicesByValuationRequest - No longer applicable with
new structure
// Use ValuationRequestRepository.GetValuationRequestsByInvoice instead

// UPDATED: SearchValuationInvoices searches valuation invoices by invoice number
func (r *ValuationInvoiceRepository) SearchValuationInvoices(query string, limit,
offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
searchPattern := "%" + query + "%"
err := r.db.Where("invoice_number ILIKE ?", searchPattern).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Offset(offset).
```

```
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// InvoiceHasPayments checks if invoice has any payments
func (r *ValuationInvoiceRepository) InvoiceHasPayments(invoiceNumber string)
(bool, error) {
var count int64
err := r.db.Model(&models.ValuationPayments{}).
Where("invoice_number = ?", invoiceNumber).
Count(&count).Error
return count > 0, err
}

// NEW: InvoiceHasRequests checks if invoice has any valuation requests assigned
func (r *ValuationInvoiceRepository) InvoiceHasRequests(invoiceID string) (bool,
error) {
var count int64
err := r.db.Model(&models.ValuationRequests{}).
Where("invoice_id = ?", invoiceID).
Count(&count).Error
return count > 0, err
}

// UPDATED: GetInvoicesByDateRange retrieves valuation invoices within a date
range
func (r *ValuationInvoiceRepository) GetInvoicesByDateRange(startDate, endDate
string, limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Where("created_at BETWEEN ? AND ?", startDate, endDate).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}
```

```
// UPDATED: GetInvoicesByAmountRange retrieves invoices within a specified amount range
func (r *ValuationInvoiceRepository) GetInvoicesByAmountRange(minAmount, maxAmount float64, limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Where("amount_billed BETWEEN ? AND ?", minAmount, maxAmount).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// UPDATED: GetOverdueInvoices retrieves invoices that are past due
func (r *ValuationInvoiceRepository) GetOverdueInvoices(daysOverdue int, limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
cutoffDate := time.Now().AddDate(0, 0, -daysOverdue)
err := r.db.Where("status = ? AND created_at < ?", models.StatusUnpaid, cutoffDate).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at ASC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// GetValuationInvoicesCountByStatus returns count of valuation invoices by status
func (r *ValuationInvoiceRepository) GetValuationInvoicesCountByStatus(status string) (int64, error) {
var count int64
err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ?", status).
```

```
Count(&count).Error
return count, err
}

// GetTotalRevenueByDateRange calculates total revenue within a date range
func (r *ValuationInvoiceRepository) GetTotalRevenueByDateRange(startDate,
endDate string) (float64, error) {
var totalRevenue float64
err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ? AND created_at BETWEEN ? AND ?", models.StatusPaid, startDate,
endDate).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&totalRevenue).Error
return totalRevenue, err
}

// GetMonthlyRevenue calculates total revenue for a specific month and year
func (r *ValuationInvoiceRepository) GetMonthlyRevenue(year, month int) (float64,
error) {
var totalRevenue float64
err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ? AND EXTRACT(YEAR FROM created_at) = ? AND EXTRACT(MONTH
FROM created_at) = ?",
models.StatusPaid, year, month).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&totalRevenue).Error
return totalRevenue, err
}

// GetTotalUnpaidAmount calculates total amount of unpaid invoices
func (r *ValuationInvoiceRepository) GetTotalUnpaidAmount() (float64, error) {
var totalUnpaid float64
err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ?", models.StatusUnpaid).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&totalUnpaid).Error
return totalUnpaid, err
}

// GetAverageInvoiceAmount calculates average invoice amount
func (r *ValuationInvoiceRepository) GetAverageInvoiceAmount() (float64, error) {
var averageAmount float64
err := r.db.Model(&models.ValuationInvoices{}).
Select("COALESCE(AVG(amount_billed), 0)").
```

```
Scan(&averageAmount).Error
return averageAmount, err
}

// BulkUpdateInvoiceStatus updates status for multiple invoices
func (r *ValuationInvoiceRepository) BulkUpdateInvoiceStatus(invoiceIDs []string,
status string) error {
return r.db.Model(&models.ValuationInvoices{}).
Where("id IN ?", invoiceIDs).
Update("status", status).Error
}

// UPDATED: GetRecentInvoices retrieves the most recent valuation invoices
func (r *ValuationInvoiceRepository) GetRecentInvoices(limit int)
([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Order("created_at DESC").
Limit(limit).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// UPDATED: GetInvoiceStatistics returns various statistics about valuation invoices
func (r *ValuationInvoiceRepository) GetInvoiceStatistics() (map[string]interface{},
error) {
stats := make(map[string]interface{})

// Total invoices
var totalCount int64
if err := r.db.Model(&models.ValuationInvoices{}).Count(&totalCount).Error; err != nil {
return nil, err
}
stats["total_invoices"] = totalCount

// Paid invoices count
var paidCount int64
```

```
if err := r.db.Model(&models.ValuationInvoices{}).Where("status = ?",
models.StatusPaid).Count(&paidCount).Error; err != nil {
return nil, err
}
stats["paid_invoices"] = paidCount

// Unpaid invoices count
var unpaidCount int64
if err := r.db.Model(&models.ValuationInvoices{}).Where("status = ?",
models.StatusUnpaid).Count(&unpaidCount).Error; err != nil {
return nil, err
}
stats["unpaid_invoices"] = unpaidCount

// Total revenue (paid invoices)
var totalRevenue float64
if err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ?", models.StatusPaid).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&totalRevenue).Error; err != nil {
return nil, err
}
stats["total_revenue"] = totalRevenue

// Total outstanding (unpaid invoices)
var totalOutstanding float64
if err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ?", models.StatusUnpaid).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&totalOutstanding).Error; err != nil {
return nil, err
}
stats["total_outstanding"] = totalOutstanding

// Average invoice amount
var averageAmount float64
if err := r.db.Model(&models.ValuationInvoices{}).
Select("COALESCE(AVG(amount_billed), 0)").
Scan(&averageAmount).Error; err != nil {
return nil, err
}
stats["average_invoice_amount"] = averageAmount

// NEW: Average requests per invoice
```

```
var avgRequestsPerInvoice float64
if err := r.db.Raw(`
SELECT COALESCE(AVG(request_count), 0) as avg_requests
FROM (
SELECT COUNT(vr.id) as request_count
FROM valuation_invoices vi
LEFT JOIN valuation_requests vr ON vr.invoice_id = vi.id
WHERE vi.deleted_at IS NULL
GROUP BY vi.id
) as invoice_request_counts
`).Scan(&avgRequestsPerInvoice).Error; err != nil {
return nil, err
}
stats["average_requests_per_invoice"] = avgRequestsPerInvoice

// NEW: Total valuation requests invoiced
var totalRequestsInvoiced int64
if err := r.db.Model(&models.ValuationRequests{}).
Where("invoice_id IS NOT NULL").
Count(&totalRequestsInvoiced).Error; err != nil {
return nil, err
}
stats["total_requests_invoiced"] = totalRequestsInvoiced

// NEW: Empty invoices (invoices with no requests)
var emptyInvoicesCount int64
if err := r.db.Raw(`
SELECT COUNT(*)
FROM valuation_invoices vi
LEFT JOIN valuation_requests vr ON vr.invoice_id = vi.id
WHERE vi.deleted_at IS NULL AND vr.id IS NULL
`).Scan(&emptyInvoicesCount).Error; err != nil {
return nil, err
}
stats["empty_invoices"] = emptyInvoicesCount

// Overdue invoices (unpaid for more than 30 days)
var overdueCount int64
cutoffDate := time.Now().AddDate(0, 0, -30)
if err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ? AND created_at < ?", models.StatusUnpaid, cutoffDate).
Count(&overdueCount).Error; err != nil {
return nil, err
}
```

```
stats["overdue_invoices"] = overdueCount

// This month's revenue
currentYear, currentMonth, _ := time.Now().Date()
var thisMonthRevenue float64
if err := r.db.Model(&models.ValuationInvoices{}).
Where("status = ? AND EXTRACT(YEAR FROM created_at) = ? AND EXTRACT(MONTH
FROM created_at) = ?",
models.StatusPaid, currentYear, int(currentMonth)).
Select("COALESCE(SUM(amount_billed), 0)").
Scan(&thisMonthRevenue).Error; err != nil {
return nil, err
}
stats["this_month_revenue"] = thisMonthRevenue

// This month's invoice count
var thisMonthCount int64
if err := r.db.Model(&models.ValuationInvoices{}).
Where("EXTRACT(YEAR FROM created_at) = ? AND EXTRACT(MONTH FROM created_at)
= ?",
currentYear, int(currentMonth)).
Count(&thisMonthCount).Error; err != nil {
return nil, err
}
stats["this_month_invoices"] = thisMonthCount

return stats, nil
}

// UPDATED: GetInvoicesByCustomer retrieves invoices for a specific customer
through valuation requests
func (r *ValuationInvoiceRepository) GetInvoicesByCustomer(customerID string,
limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
Joins("JOIN valuation_requests ON valuation_requests.invoice_id =
valuation_invoices.id").
Where("valuation_requests.customer_id = ?", customerID).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
```

```
Distinct("valuation_invoices.id").
Order("valuation_invoices.created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// UPDATED: GetInvoicesByPartner retrieves invoices for a specific partner through
valuation requests
func (r *ValuationInvoiceRepository) GetInvoicesByPartner(partnerID string, limit,
offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
Joins("JOIN valuation_requests ON valuation_requests.invoice_id =
valuation_invoices.id").
Where("valuation_requests.partner_id = ?", partnerID).
Preload("ValuationRequests"). // Load multiple requests
Preload("ValuationRequests.Customer"). // Load customer for each request
Preload("ValuationRequests.Partner"). // Load partner for each request
Preload("ValuationRequests.Agent"). // Load agent for each request
Preload("ValuationRequests.Valuer"). // Load valuer for each request
// REMOVED: Preload("ValuationPayments") - this was causing the error
Distinct("valuation_invoices.id").
Order("valuation_invoices.created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// ADD: Method to get payments for an invoice (since we can't preload)
func (r *ValuationInvoiceRepository) GetPaymentsForInvoice(invoiceNumber string)
([]models.ValuationPayments, error) {
var payments []models.ValuationPayments
err := r.db.Where("invoice_number = ?", invoiceNumber).
Order("created_at DESC").
Find(&payments).Error
return payments, err
}

// ADD: Method to get invoice with payments loaded manually
func (r *ValuationInvoiceRepository) GetInvoiceWithPayments(invoiceID string)
(*models.ValuationInvoices, []models.ValuationPayments, error) {
```

```
// Get the invoice
invoice, err := r.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, nil, err
}

// Get the payments separately
payments, err := r.GetPaymentsForInvoice(invoice.InvoiceNumber)
if err != nil {
return nil, nil, err
}

return invoice, payments, nil
}

// NEW: GetInvoicesByAgent retrieves invoices for a specific agent through valuation
requests
func (r *ValuationInvoiceRepository) GetInvoicesByAgent(agentID string, limit, offset
int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
Joins("JOIN valuation_requests ON valuation_requests.invoice_id =
valuation_invoices.id").
Where("valuation_requests.agent_id = ?", agentID).
Preload("ValuationRequests").
Preload("ValuationRequests.Customer").
Preload("ValuationRequests.Partner").
Preload("ValuationRequests.Agent").
Preload("ValuationRequests.Valuer").
//Preload("ValuationPayments").
Distinct("valuation_invoices.id").
Order("valuation_invoices.created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// NEW: GetInvoicesByValuer retrieves invoices for a specific valuer through valuation
requests
func (r *ValuationInvoiceRepository) GetInvoicesByValuer(valuerID string, limit, offset
int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.
```

```
Joins("JOIN valuation_requests ON valuation_requests.invoice_id =
valuation_invoices.id").
Where("valuation_requests.valuer_id = ?", valuerID).
Preload("ValuationRequests").
Preload("ValuationRequests.Customer").
Preload("ValuationRequests.Partner").
Preload("ValuationRequests.Agent").
Preload("ValuationRequests.Valuer").
//Preload("ValuationPayments").
Distinct("valuation_invoices.id").
Order("valuation_invoices.created_at DESC").
Limit(limit).
Offset(offset).
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// NEW: GetInvoicesWithMultipleRequests retrieves invoices that have multiple
valuation requests
func (r *ValuationInvoiceRepository) GetInvoicesWithMultipleRequests(limit, offset
int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Raw(`
SELECT vi.* FROM valuation_invoices vi
WHERE vi.id IN (
SELECT invoice_id
FROM valuation_requests
WHERE invoice_id IS NOT NULL
GROUP BY invoice_id
HAVING COUNT(*) > 1
)
AND vi.deleted_at IS NULL
ORDER BY vi.created_at DESC
LIMIT ? OFFSET ?
`, limit, offset).
Preload("ValuationRequests").
Preload("ValuationRequests.Customer").
Preload("ValuationRequests.Partner").
Preload("ValuationRequests.Agent").
Preload("ValuationRequests.Valuer").
//Preload("ValuationPayments").
Find(&valuationInvoices).Error
return valuationInvoices, err
}
```

```
// NEW: GetInvoicesWithSingleRequest retrieves invoices that have only one valuation request
func (r *ValuationInvoiceRepository) GetInvoicesWithSingleRequest(limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Raw(`
SELECT vi.* FROM valuation_invoices vi
WHERE vi.id IN (
SELECT invoice_id
FROM valuation_requests
WHERE invoice_id IS NOT NULL
GROUP BY invoice_id
HAVING COUNT(*) = 1
)
AND vi.deleted_at IS NULL
ORDER BY vi.created_at DESC
LIMIT ? OFFSET ?
`, limit, offset).
Preload("ValuationRequests").
Preload("ValuationRequests.Customer").
Preload("ValuationRequests.Partner").
Preload("ValuationRequests.Agent").
Preload("ValuationRequests.Valuer").
//Preload("ValuationPayments").
Find(&valuationInvoices).Error
return valuationInvoices, err
}

// NEW: GetEmptyInvoices retrieves invoices that have no valuation requests assigned
func (r *ValuationInvoiceRepository) GetEmptyInvoices(limit, offset int) ([]models.ValuationInvoices, error) {
var valuationInvoices []models.ValuationInvoices
err := r.db.Raw(`
SELECT vi.* FROM valuation_invoices vi
LEFT JOIN valuation_requests vr ON vr.invoice_id = vi.id
WHERE vi.deleted_at IS NULL AND vr.id IS NULL
ORDER BY vi.created_at DESC
LIMIT ? OFFSET ?
`, limit, offset).
Preload("ValuationPayments"). // Only load payments since there are no requests
Find(&valuationInvoices).Error
return valuationInvoices, err
```

```
}

// NEW: GetInvoiceRequestCount returns the number of requests for a specific
invoice
func (r *ValuationInvoiceRepository) GetInvoiceRequestCount(invoiceID string) (int64,
error) {
var count int64
err := r.db.Model(&models.ValuationRequests{}).
Where("invoice_id = ?", invoiceID).
Count(&count).Error
return count, err
}


// valuation_invoice_service.go
package services

import (
"fmt"
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/
rabbitmq"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/repositories"
"github.com/google/uuid"
)

type ValuationInvoiceService struct {
valuationInvoiceRepo *repositories.ValuationInvoiceRepository
valuationRequestRepo *repositories.ValuationRequestRepository
rabbitmqProducer *rabbitmq.Producer
}

func NewValuationInvoiceService(
valuationInvoiceRepo *repositories.ValuationInvoiceRepository,
valuationRequestRepo *repositories.ValuationRequestRepository,
rabbitmqProducer *rabbitmq.Producer,
) *ValuationInvoiceService {
return &ValuationInvoiceService{
valuationInvoiceRepo: valuationInvoiceRepo,
valuationRequestRepo: valuationRequestRepo,
```

```
rabbitmqProducer: rabbitmqProducer,
}
}

// UPDATED: Request/Response structs for new system
type CreateValuationInvoiceRequest struct {
InvoiceNumber string `json:"invoice_number" validate:"required"`
AmountBilled float64 `json:"amount_billed" validate:"required,gt=0"`
Status string `json:"status,omitempty"`
ValuationRequestIDs []string `json:"valuation_request_ids,omitempty"` // NEW:
Optional initial requests
}

type UpdateValuationInvoiceRequest struct {
InvoiceNumber *string `json:"invoice_number,omitempty"`
AmountBilled *float64 `json:"amount_billed,omitempty" validate:"omitempty,gt=0"`
Status *string `json:"status,omitempty"`
}

type UpdateInvoiceStatusRequest struct {
Status string `json:"status" validate:"required"`
}

// NEW: Bulk invoice creation request
type BulkCreateInvoiceRequest1 struct {
ValuationRequestIDs []string `json:"valuation_request_ids" validate:"required,min=1"`
InvoiceNumber string `json:"invoice_number" validate:"required"`
AmountBilled float64 `json:"amount_billed" validate:"required,gt=0"`
Status string `json:"status,omitempty"`
}

// NEW: Add requests to existing invoice
type AddRequestsToInvoiceRequest struct {
ValuationRequestIDs []string `json:"valuation_request_ids" validate:"required,min=1"`
}

// UPDATED: CreateValuationInvoice creates a new valuation invoice (with optional
requests)
func (s *ValuationInvoiceService) CreateValuationInvoice(req
CreateValuationInvoiceRequest) (*models.ValuationInvoices, error) {
// Check if invoice number already exists
existing, err :=
s.valuationInvoiceRepo.GetValuationInvoiceByInvoiceNumber(req.InvoiceNumber)
if err == nil && existing != nil {
```

```
return nil, fmt.Errorf("invoice with number %s already exists", req.InvoiceNumber)
}

// Validate valuation requests if provided
if len(req.ValuationRequestIDs) > 0 {
for _, requestID := range req.ValuationRequestIDs {
valuationRequest, err :=
s.valuationRequestRepo.GetValuationRequestByID(requestID)
if err != nil {
return nil, fmt.Errorf("valuation request %s not found: %v", requestID, err)
}

// if valuationRequest.Status != models.StatusApproved {
// return nil, fmt.Errorf("can only create invoices for approved valuation requests
(request %s is %s)", requestID, valuationRequest.Status)
// }

if valuationRequest.InvoiceID != nil {
return nil, fmt.Errorf("valuation request %s is already invoiced", requestID)
}
}
}

// Set default status if not provided
status := req.Status
if status == "" {
status = string(models.StatusUnpaid)
}

// Create valuation invoice
valuationInvoice := &models.ValuationInvoices{
InvoiceNumber: req.InvoiceNumber,
AmountBilled: req.AmountBilled,
Status: models.StatusType(status),
}

if err := s.valuationInvoiceRepo.CreateValuationInvoice(valuationInvoice); err != nil {
return nil, fmt.Errorf("failed to create valuation invoice: %v", err)
}

// Assign requests to the invoice if provided
if len(req.ValuationRequestIDs) > 0 {
```

```
if err :=
s.valuationRequestRepo.BulkAssignInvoiceToRequests(req.ValuationRequestIDs,
valuationInvoice.ID); err != nil {
return nil, fmt.Errorf("failed to assign requests to invoice: %v", err)
}
}

// Load relationships for response
valuationInvoice, err =
s.valuationInvoiceRepo.GetValuationInvoiceByID(valuationInvoice.ID)
if err != nil {
return nil, fmt.Errorf("failed to load created valuation invoice: %v", err)
}

// Publish valuation invoice created event
s.publishValuationInvoiceEvent("valuation_invoice.created", valuationInvoice)

return valuationInvoice, nil
}

// NEW: BulkCreateInvoiceForRequests creates an invoice for multiple valuation
requests
func (s *ValuationInvoiceService) BulkCreateInvoiceForRequests(req
BulkCreateInvoiceRequest1) (*models.ValuationInvoices, error) {
createReq := CreateValuationInvoiceRequest{
InvoiceNumber: req.InvoiceNumber,
AmountBilled: req.AmountBilled,
Status: req.Status,
ValuationRequestIDs: req.ValuationRequestIDs,
}

return s.CreateValuationInvoice(createReq)
}

// NEW: AddRequestsToInvoice adds valuation requests to an existing invoice
func (s *ValuationInvoiceService) AddRequestsToInvoice(invoiceID string, req
AddRequestsToInvoiceRequest) (*models.ValuationInvoices, error) {
// Check if invoice exists
invoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("invoice not found: %v", err)
}

// Prevent adding requests to paid invoices
```

```
if invoice.Status == models.StatusPaid {
return nil, fmt.Errorf("cannot add requests to paid invoices")
}

// Validate all requests exist and can be added
for _, requestID := range req.ValuationRequestIDs {
valuationRequest, err :=
s.valuationRequestRepo.GetValuationRequestByID(requestID)
if err != nil {
return nil, fmt.Errorf("valuation request %s not found: %v", requestID, err)
}

if valuationRequest.Status != models.StatusApproved {
return nil, fmt.Errorf("can only add approved valuation requests to invoices (request
%s is %s)", requestID, valuationRequest.Status)
}

if valuationRequest.InvoiceID != nil {
return nil, fmt.Errorf("valuation request %s is already invoiced", requestID)
}
}

// Assign requests to invoice
if err :=
s.valuationRequestRepo.BulkAssignInvoiceToRequests(req.ValuationRequestIDs,
invoiceID); err != nil {
return nil, fmt.Errorf("failed to assign requests to invoice: %v", err)
}

// Reload with updated relationships
invoice, err = s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("failed to load updated invoice: %v", err)
}

// Publish requests added event
s.publishValuationInvoiceEvent("valuation_invoice.requests_added", invoice)

return invoice, nil
}

// NEW: RemoveRequestsFromInvoice removes valuation requests from an invoice
func (s *ValuationInvoiceService) RemoveRequestsFromInvoice(invoiceID string,
requestIDs []string) (*models.ValuationInvoices, error) {
```

```
// Check if invoice exists
invoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("invoice not found: %v", err)
}

// Prevent removing requests from paid invoices
if invoice.Status == models.StatusPaid {
return nil, fmt.Errorf("cannot remove requests from paid invoices")
}

// Validate all requests belong to this invoice
for _, requestID := range requestIDs {
valuationRequest, err :=
s.valuationRequestRepo.GetValuationRequestByID(requestID)
if err != nil {
return nil, fmt.Errorf("valuation request %s not found: %v", requestID, err)
}

if valuationRequest.InvoiceID == nil || *valuationRequest.InvoiceID != invoiceID {
return nil, fmt.Errorf("valuation request %s does not belong to this invoice",
requestID)
}
}

// Remove requests from invoice
if err := s.valuationRequestRepo.RemoveInvoiceFromRequests(requestIDs); err != nil {
return nil, fmt.Errorf("failed to remove requests from invoice: %v", err)
}

// Reload with updated relationships
invoice, err = s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("failed to load updated invoice: %v", err)
}

// Publish requests removed event
s.publishValuationInvoiceEvent("valuation_invoice.requests_removed", invoice)

return invoice, nil
}

// GetAllValuationInvoices retrieves all valuation invoices with pagination
```

```
func (s *ValuationInvoiceService) GetAllValuationInvoices(limit, offset int)
([]models.ValuationInvoices, int64, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetAllValuationInvoices(limit, offset)
if err != nil {
return nil, 0, fmt.Errorf("failed to retrieve valuation invoices: %v", err)
}

total, err := s.valuationInvoiceRepo.GetValuationInvoicesCount()
if err != nil {
return nil, 0, fmt.Errorf("failed to get valuation invoices count: %v", err)
}

return valuationInvoices, total, nil
}

// GetValuationInvoiceByID retrieves a valuation invoice by ID
func (s *ValuationInvoiceService) GetValuationInvoiceByID(invoiceID string)
(*models.ValuationInvoices, error) {
valuationInvoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("valuation invoice not found: %v", err)
}

return valuationInvoice, nil
}

// GetValuationInvoiceByInvoiceNumber retrieves a valuation invoice by invoice
number
func (s *ValuationInvoiceService)
GetValuationInvoiceByInvoiceNumber(invoiceNumber string)
(*models.ValuationInvoices, error) {
valuationInvoice, err :=
s.valuationInvoiceRepo.GetValuationInvoiceByInvoiceNumber(invoiceNumber)
if err != nil {
return nil, fmt.Errorf("valuation invoice not found: %v", err)
}

return valuationInvoice, nil
}

// UPDATED: UpdateValuationInvoice updates an existing valuation invoice
func (s *ValuationInvoiceService) UpdateValuationInvoice(invoiceID string, req
UpdateValuationInvoiceRequest) (*models.ValuationInvoices, error) {
// Get existing valuation invoice
```

```
valuationInvoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("valuation invoice not found: %v", err)
}

// Prevent updating paid invoices
if valuationInvoice.Status == models.StatusPaid {
return nil, fmt.Errorf("cannot update paid invoices")
}

// Check for invoice number conflicts if being updated
if req.InvoiceNumber != nil && *req.InvoiceNumber !=
valuationInvoice.InvoiceNumber {
existing, err :=
s.valuationInvoiceRepo.GetValuationInvoiceByInvoiceNumber(*req.InvoiceNumber)
if err == nil && existing != nil && existing.ID != invoiceID {
return nil, fmt.Errorf("invoice with number %s already exists", *req.InvoiceNumber)
}
valuationInvoice.InvoiceNumber = *req.InvoiceNumber
}

// Update other fields
if req.AmountBilled != nil {
valuationInvoice.AmountBilled = *req.AmountBilled
}
if req.Status != nil {
valuationInvoice.Status = models.StatusType(*req.Status)
}

// Save updates
if err := s.valuationInvoiceRepo.UpdateValuationInvoice(valuationInvoice); err != nil {
return nil, fmt.Errorf("failed to update valuation invoice: %v", err)
}

// Reload with relationships
valuationInvoice, err = s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("failed to load updated valuation invoice: %v", err)
}

// Publish valuation invoice updated event
s.publishValuationInvoiceEvent("valuation_invoice.updated", valuationInvoice)

return valuationInvoice, nil
```

```
}

// UPDATED: DeleteValuationInvoice soft deletes a valuation invoice
func (s *ValuationInvoiceService) DeleteValuationInvoice(invoiceID string) error {
// Check if valuation invoice exists
valuationInvoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return fmt.Errorf("valuation invoice not found: %v", err)
}

// Prevent deleting paid invoices
if valuationInvoice.Status == models.StatusPaid {
return fmt.Errorf("cannot delete paid invoices")
}

// Check if invoice has any payments
hasPayments, err :=
s.valuationInvoiceRepo.InvoiceHasPayments(valuationInvoice.InvoiceNumber)
if err != nil {
return fmt.Errorf("failed to check invoice dependencies: %v", err)
}

if hasPayments {
return fmt.Errorf("cannot delete invoice with existing payments")
}

// NEW: Check if invoice has valuation requests and unassign them
hasRequests, err := s.valuationInvoiceRepo.InvoiceHasRequests(invoiceID)
if err != nil {
return fmt.Errorf("failed to check invoice requests: %v", err)
}

if hasRequests {
// Get all request IDs for this invoice
requests, err := s.valuationRequestRepo.GetValuationRequestsByInvoice(invoiceID,
1000, 0) // Get all requests
if err != nil {
return fmt.Errorf("failed to get invoice requests: %v", err)
}

requestIDs := make([]string, len(requests))
for i, request := range requests {
requestIDs[i] = request.ID
}
```

```
// Remove invoice assignment from all requests
if err := s.valuationRequestRepo.RemoveInvoiceFromRequests(requestIDs); err != nil {
return fmt.Errorf("failed to unassign requests from invoice: %v", err)
}
}

// Soft delete valuation invoice
if err := s.valuationInvoiceRepo.DeleteValuationInvoice(invoiceID); err != nil {
return fmt.Errorf("failed to delete valuation invoice: %v", err)
}

// Publish valuation invoice deleted event
s.publishValuationInvoiceEvent("valuation_invoice.deleted", valuationInvoice)

return nil
}

// GetValuationInvoicesByStatus retrieves valuation invoices by status
func (s *ValuationInvoiceService) GetValuationInvoicesByStatus(status string, limit,
offset int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetValuationInvoicesByStatus(status,
limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve valuation invoices by status: %v", err)
}

return valuationInvoices, nil
}

// REMOVED: GetValuationInvoicesByValuationRequest - No longer applicable with
new structure

// UpdateInvoiceStatus updates the status of a valuation invoice
func (s *ValuationInvoiceService) UpdateInvoiceStatus(invoiceID, status string)
(*models.ValuationInvoices, error) {
// Get existing valuation invoice
valuationInvoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("valuation invoice not found: %v", err)
}

// Business rule: prevent changing from paid to unpaid
```

```
if valuationInvoice.Status == models.StatusPaid && status ==
string(models.StatusUnpaid) {
return nil, fmt.Errorf("cannot change paid invoice back to unpaid")
}

// Update status
valuationInvoice.Status = models.StatusType(status)

// Save updates
if err := s.valuationInvoiceRepo.UpdateValuationInvoice(valuationInvoice); err != nil {
return nil, fmt.Errorf("failed to update valuation invoice status: %v", err)
}

// Reload with relationships
valuationInvoice, err = s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return nil, fmt.Errorf("failed to load updated valuation invoice: %v", err)
}

// Publish status updated event
s.publishValuationInvoiceEvent("valuation_invoice.status_updated", valuationInvoice)

return valuationInvoice, nil
}

// UpdateInvoiceStatusByInvoiceNumber updates the status of a valuation invoice by
invoice number
func (s *ValuationInvoiceService)
UpdateInvoiceStatusByInvoiceNumber(invoiceNumber, status string)
(*models.ValuationInvoices, error) {
// Get existing valuation invoice by invoice number
valuationInvoice, err :=
s.valuationInvoiceRepo.GetValuationInvoiceByInvoiceNumber(invoiceNumber)
if err != nil {
return nil, fmt.Errorf("valuation invoice not found: %v", err)
}

return s.UpdateInvoiceStatus(valuationInvoice.ID, status)
}

// SearchValuationInvoices searches valuation invoices by invoice number
func (s *ValuationInvoiceService) SearchValuationInvoices(query string, limit, offset
int) ([]models.ValuationInvoices, error) {
```

```
valuationInvoices, err := s.valuationInvoiceRepo.SearchValuationInvoices(query, limit,
offset)
if err != nil {
return nil, fmt.Errorf("failed to search valuation invoices: %v", err)
}

return valuationInvoices, nil
}

// GetPaidInvoices retrieves all paid valuation invoices
func (s *ValuationInvoiceService) GetPaidInvoices(limit, offset int)
([]models.ValuationInvoices, error) {
return s.GetValuationInvoicesByStatus(string(models.StatusPaid), limit, offset)
}

// GetUnpaidInvoices retrieves all unpaid valuation invoices
func (s *ValuationInvoiceService) GetUnpaidInvoices(limit, offset int)
([]models.ValuationInvoices, error) {
return s.GetValuationInvoicesByStatus(string(models.StatusUnpaid), limit, offset)
}

// GetInvoicesByDateRange retrieves valuation invoices within a date range
func (s *ValuationInvoiceService) GetInvoicesByDateRange(startDate, endDate string,
limit, offset int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesByDateRange(startDate,
endDate, limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve valuation invoices by date range: %v", err)
}

return valuationInvoices, nil
}

// GetInvoiceStatistics returns various statistics about valuation invoices
func (s *ValuationInvoiceService) GetInvoiceStatistics() (map[string]interface{}, error)
{
statistics, err := s.valuationInvoiceRepo.GetInvoiceStatistics()
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoice statistics: %v", err)
}

return statistics, nil
}
```

```
// MarkInvoiceAsPaid marks an invoice as paid
func (s *ValuationInvoiceService) MarkInvoiceAsPaid(invoiceID string)
(*models.ValuationInvoices, error) {
return s.UpdateInvoiceStatus(invoiceID, string(models.StatusPaid))
}

// MarkInvoiceAsUnpaid marks an invoice as unpaid
func (s *ValuationInvoiceService) MarkInvoiceAsUnpaid(invoiceID string)
(*models.ValuationInvoices, error) {
return s.UpdateInvoiceStatus(invoiceID, string(models.StatusUnpaid))
}

// MarkInvoiceAsPaidByInvoiceNumber marks an invoice as paid by invoice number
func (s *ValuationInvoiceService)
MarkInvoiceAsPaidByInvoiceNumber(invoiceNumber string)
(*models.ValuationInvoices, error) {
return s.UpdateInvoiceStatusByInvoiceNumber(invoiceNumber,
string(models.StatusPaid))
}

// MarkInvoiceAsUnpaidByInvoiceNumber marks an invoice as unpaid by invoice
number
func (s *ValuationInvoiceService)
MarkInvoiceAsUnpaidByInvoiceNumber(invoiceNumber string)
(*models.ValuationInvoices, error) {
return s.UpdateInvoiceStatusByInvoiceNumber(invoiceNumber,
string(models.StatusUnpaid))
}

// GetOverdueInvoices retrieves invoices that are past due (unpaid and older than
specified days)
func (s *ValuationInvoiceService) GetOverdueInvoices(daysOverdue int, limit, offset
int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetOverdueInvoices(daysOverdue,
limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve overdue invoices: %v", err)
}

return valuationInvoices, nil
}

// GetTotalRevenueByDateRange calculates total revenue within a date range
```

```
func (s *ValuationInvoiceService) GetTotalRevenueByDateRange(startDate, endDate
string) (float64, error) {
totalRevenue, err := s.valuationInvoiceRepo.GetTotalRevenueByDateRange(startDate,
endDate)
if err != nil {
return 0, fmt.Errorf("failed to calculate total revenue: %v", err)
}

return totalRevenue, nil
}

// GetMonthlyRevenue calculates total revenue for a specific month and year
func (s *ValuationInvoiceService) GetMonthlyRevenue(year, month int) (float64, error)
{
totalRevenue, err := s.valuationInvoiceRepo.GetMonthlyRevenue(year, month)
if err != nil {
return 0, fmt.Errorf("failed to calculate monthly revenue: %v", err)
}

return totalRevenue, nil
}

// GetCurrentMonthRevenue calculates total revenue for the current month
func (s *ValuationInvoiceService) GetCurrentMonthRevenue() (float64, error) {
now := time.Now()
return s.GetMonthlyRevenue(now.Year(), int(now.Month()))
}

// GetCurrentYearRevenue calculates total revenue for the current year
func (s *ValuationInvoiceService) GetCurrentYearRevenue() (float64, error) {
now := time.Now()
startDate := fmt.Sprintf("%d-01-01", now.Year())
endDate := fmt.Sprintf("%d-12-31", now.Year())
return s.GetTotalRevenueByDateRange(startDate, endDate)
}

// GetInvoicesByAmountRange retrieves invoices within a specified amount range
func (s *ValuationInvoiceService) GetInvoicesByAmountRange(minAmount,
maxAmount float64, limit, offset int) ([]models.ValuationInvoices, error) {
valuationInvoices, err :=
s.valuationInvoiceRepo.GetInvoicesByAmountRange(minAmount, maxAmount, limit,
offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices by amount range: %v", err)
```

```
}

return valuationInvoices, nil
}

// GetTotalUnpaidAmount calculates total amount of unpaid invoices
func (s *ValuationInvoiceService) GetTotalUnpaidAmount() (float64, error) {
totalUnpaid, err := s.valuationInvoiceRepo.GetTotalUnpaidAmount()
if err != nil {
return 0, fmt.Errorf("failed to calculate total unpaid amount: %v", err)
}

return totalUnpaid, nil
}

// GetAverageInvoiceAmount calculates average invoice amount
func (s *ValuationInvoiceService) GetAverageInvoiceAmount() (float64, error) {
averageAmount, err := s.valuationInvoiceRepo.GetAverageInvoiceAmount()
if err != nil {
return 0, fmt.Errorf("failed to calculate average invoice amount: %v", err)
}

return averageAmount, nil
}

// GetInvoicesByCustomer retrieves invoices for a specific customer through
valuation requests
func (s *ValuationInvoiceService) GetInvoicesByCustomer(customerID string, limit,
offset int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesByCustomer(customerID,
limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices by customer: %v", err)
}

return valuationInvoices, nil
}

// GetInvoicesByPartner retrieves invoices for a specific partner through valuation
requests
func (s *ValuationInvoiceService) GetInvoicesByPartner(partnerID string, limit, offset
int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesByPartner(partnerID,
limit, offset)
```

```
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices by partner: %v", err)
}

return valuationInvoices, nil
}

// NEW: GetInvoicesByAgent retrieves invoices for a specific agent through valuation
requests
func (s *ValuationInvoiceService) GetInvoicesByAgent(agentID string, limit, offset int)
([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesByAgent(agentID, limit,
offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices by agent: %v", err)
}

return valuationInvoices, nil
}

// NEW: GetInvoicesByValuer retrieves invoices for a specific valuer through valuation
requests
func (s *ValuationInvoiceService) GetInvoicesByValuer(valuerID string, limit, offset
int) ([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesByValuer(valuerID, limit,
offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices by valuer: %v", err)
}

return valuationInvoices, nil
}

// GetRecentInvoices retrieves the most recent valuation invoices
func (s *ValuationInvoiceService) GetRecentInvoices(limit int)
([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetRecentInvoices(limit)
if err != nil {
return nil, fmt.Errorf("failed to retrieve recent invoices: %v", err)
}

return valuationInvoices, nil
}
```

```
// BulkUpdateInvoiceStatus updates status for multiple invoices
func (s *ValuationInvoiceService) BulkUpdateInvoiceStatus(invoiceIDs []string, status
string) error {
// Validate all invoices exist and can be updated
for _, invoiceID := range invoiceIDs {
invoice, err := s.valuationInvoiceRepo.GetValuationInvoiceByID(invoiceID)
if err != nil {
return fmt.Errorf("invoice %s not found: %v", invoiceID, err)
}

// Business rule: prevent changing from paid to unpaid
if invoice.Status == models.StatusPaid && status == string(models.StatusUnpaid) {
return fmt.Errorf("cannot change paid invoice %s back to unpaid", invoiceID)
}
}

// Perform bulk update
if err := s.valuationInvoiceRepo.BulkUpdateInvoiceStatus(invoiceIDs, status); err != nil
{
return fmt.Errorf("failed to bulk update invoice status: %v", err)
}

// Publish bulk update event
s.publishBulkInvoiceEvent("valuation_invoice.bulk_status_updated", invoiceIDs,
status)

return nil
}

// BulkMarkInvoicesAsPaid marks multiple invoices as paid
func (s *ValuationInvoiceService) BulkMarkInvoicesAsPaid(invoiceIDs []string) error {
return s.BulkUpdateInvoiceStatus(invoiceIDs, string(models.StatusPaid))
}

// BulkMarkInvoicesAsUnpaid marks multiple invoices as unpaid
func (s *ValuationInvoiceService) BulkMarkInvoicesAsUnpaid(invoiceIDs []string)
error {
return s.BulkUpdateInvoiceStatus(invoiceIDs, string(models.StatusUnpaid))
}

// GetInvoicesCountByStatus returns count of invoices by status
func (s *ValuationInvoiceService) GetInvoicesCountByStatus(status string) (int64,
error) {
count, err := s.valuationInvoiceRepo.GetValuationInvoicesCountByStatus(status)
```

```
if err != nil {
return 0, fmt.Errorf("failed to get invoices count by status: %v", err)
}

return count, nil
}

// GetPaidInvoicesCount returns count of paid invoices
func (s *ValuationInvoiceService) GetPaidInvoicesCount() (int64, error) {
return s.GetInvoicesCountByStatus(string(models.StatusPaid))
}

// GetUnpaidInvoicesCount returns count of unpaid invoices
func (s *ValuationInvoiceService) GetUnpaidInvoicesCount() (int64, error) {
return s.GetInvoicesCountByStatus(string(models.StatusUnpaid))
}

// NEW: GetInvoicesWithMultipleRequests retrieves invoices that have multiple
valuation requests
func (s *ValuationInvoiceService) GetInvoicesWithMultipleRequests(limit, offset int)
([]models.ValuationInvoices, error) {
valuationInvoices, err :=
s.valuationInvoiceRepo.GetInvoicesWithMultipleRequests(limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices with multiple requests: %v", err)
}

return valuationInvoices, nil
}

// NEW: GetInvoicesWithSingleRequest retrieves invoices that have only one valuation
request
func (s *ValuationInvoiceService) GetInvoicesWithSingleRequest(limit, offset int)
([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetInvoicesWithSingleRequest(limit,
offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve invoices with single request: %v", err)
}

return valuationInvoices, nil
}
```

```
// NEW: GetEmptyInvoices retrieves invoices that have no valuation requests
assigned
func (s *ValuationInvoiceService) GetEmptyInvoices(limit, offset int)
([]models.ValuationInvoices, error) {
valuationInvoices, err := s.valuationInvoiceRepo.GetEmptyInvoices(limit, offset)
if err != nil {
return nil, fmt.Errorf("failed to retrieve empty invoices: %v", err)
}

return valuationInvoices, nil
}

// NEW: GetInvoiceRequestCount returns the number of requests for a specific
invoice
func (s *ValuationInvoiceService) GetInvoiceRequestCount(invoiceID string) (int64,
error) {
count, err := s.valuationInvoiceRepo.GetInvoiceRequestCount(invoiceID)
if err != nil {
return 0, fmt.Errorf("failed to get invoice request count: %v", err)
}

return count, nil
}

// REMOVED: AutoGenerateInvoiceForRequest - No longer applicable with new
structure
// Use BulkCreateInvoiceForRequests instead

// ProcessPaymentEvent handles payment events to update invoice status
func (s *ValuationInvoiceService) ProcessPaymentEvent(event rabbitmq.Event) error {
switch event.Type {
case "payment.completed":
if invoiceNumber, ok := event.Data["invoice_number"].(string); ok {
_, err := s.MarkInvoiceAsPaidByInvoiceNumber(invoiceNumber)
if err != nil {
return fmt.Errorf("failed to mark invoice as paid: %v", err)
}
}
case "payment.failed":
if _, ok := event.Data["invoice_number"].(string); ok {
// Keep invoice as unpaid, optionally log the failure
return nil
}
case "payment.refunded":
```

```
if invoiceNumber, ok := event.Data["invoice_number"].(string); ok {
_, err := s.MarkInvoiceAsUnpaidByInvoiceNumber(invoiceNumber)
if err != nil {
return fmt.Errorf("failed to mark invoice as unpaid after refund: %v", err)
}
}
}

return nil
}

// Utility methods
func (s *ValuationInvoiceService) generateInvoiceNumber() string {
timestamp := time.Now().Unix()
return fmt.Sprintf("INV-%d-%s", timestamp, uuid.New().String()[:8])
}

func (s *ValuationInvoiceService) publishValuationInvoiceEvent(eventType string,
valuationInvoice *models.ValuationInvoices) {
if s.rabbitmqProducer == nil {
return
}

// NEW: Include request information in events
requestCount := len(valuationInvoice.ValuationRequests)
var requestIDs []string
var customerIDs []string
for _, request := range valuationInvoice.ValuationRequests {
requestIDs = append(requestIDs, request.ID)
customerIDs = append(customerIDs, request.CustomerID)
}

event := rabbitmq.Event{
ID: uuid.New().String(),
Type: eventType,
Source: "valuation-service",
Data: map[string]interface{}{
"invoice_id": valuationInvoice.ID,
"invoice_number": valuationInvoice.InvoiceNumber,
"amount_billed": valuationInvoice.AmountBilled,
"status": valuationInvoice.Status,
"request_count": requestCount, // NEW
"valuation_request_ids": requestIDs, // NEW
"customer_ids": customerIDs, // NEW
```

```
},
Timestamp: time.Now().Unix(),
Version: "1.0",
}

s.rabbitmqProducer.PublishEvent("valuation_events", event)
}

func (s *ValuationInvoiceService) publishBulkInvoiceEvent(eventType string,
invoiceIDs []string, status string) {
if s.rabbitmqProducer == nil {
return
}

event := rabbitmq.Event{
ID: uuid.New().String(),
Type: eventType,
Source: "valuation-service",
Data: map[string]interface{}{
"invoice_ids": invoiceIDs,
"status": status,
},
Timestamp: time.Now().Unix(),
Version: "1.0",
}

s.rabbitmqProducer.PublishEvent("valuation_events", event)
} // valuation_invoice_service.go


// valuation_invoice_handler.go
package handlers

import (
"fmt"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-
service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type ValuationInvoiceHandler struct {
```

```
valuationInvoiceService *services.ValuationInvoiceService
validator *validator.Validate
}

func NewValuationInvoiceHandler(valuationInvoiceService
*services.ValuationInvoiceService) *ValuationInvoiceHandler {
return &ValuationInvoiceHandler{
valuationInvoiceService: valuationInvoiceService,
validator: validator.New(),
}
}

// CreateValuationInvoice godoc
// @Summary Create a new valuation invoice
// @Description Create a new valuation invoice in the system (optionally with
valuation requests)
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param request body services.CreateValuationInvoiceRequest true "Create
valuation invoice request"
// @Success 201 {object} response.Response "Valuation invoice created successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices [post]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) CreateValuationInvoice(c *gin.Context) {
var req services.CreateValuationInvoiceRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err := h.valuationInvoiceService.CreateValuationInvoice(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}
```

```
response.Success(c, "Valuation invoice created successfully", valuationInvoice)
}

// BulkCreateInvoiceForRequests godoc
// @Summary Create invoice for multiple requests
// @Description Create a new valuation invoice for multiple valuation requests
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param request body services.BulkCreateInvoiceRequest true "Bulk create invoice
request"
// @Success 201 {object} response.Response "Invoice created for requests
successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/bulk-create [post]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) BulkCreateInvoiceForRequests(c *gin.Context) {
var req services.BulkCreateInvoiceRequest1
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err := h.valuationInvoiceService.BulkCreateInvoiceForRequests(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

data := map[string]interface{}{
"invoice": valuationInvoice,
"requests_assigned": len(req.ValuationRequestIDs),
"request_ids": req.ValuationRequestIDs,
}

response.Success(c, "Invoice created for requests successfully", data)
}
```

```
// AddRequestsToInvoice godoc
// @Summary Add requests to existing invoice
// @Description Add valuation requests to an existing invoice
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param id path string true "Invoice ID"
// @Param request body services.AddRequestsToInvoiceRequest true "Add requests
to invoice"
// @Success 200 {object} response.Response "Requests added to invoice successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/{id}/add-requests [patch]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) AddRequestsToInvoice(c *gin.Context) {
invoiceID := c.Param("id")

var req services.AddRequestsToInvoiceRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err := h.valuationInvoiceService.AddRequestsToInvoice(invoiceID,
req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

data := map[string]interface{}{
"invoice": valuationInvoice,
"requests_added": len(req.ValuationRequestIDs),
"request_ids": req.ValuationRequestIDs,
"total_requests": len(valuationInvoice.ValuationRequests),
}

response.Success(c, "Requests added to invoice successfully", data)
}
```

```
// RemoveRequestsFromInvoice godoc
// @Summary Remove requests from invoice
// @Description Remove valuation requests from an invoice
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param id path string true "Invoice ID"
// @Param request body map[string][]string true "Request IDs to remove"
// @Success 200 {object} response.Response "Requests removed from invoice
successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/{id}/remove-requests [patch]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) RemoveRequestsFromInvoice(c *gin.Context) {
invoiceID := c.Param("id")

var req struct {
ValuationRequestIDs []string `json:"valuation_request_ids" validate:"required,min=1"`
}

if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err :=
h.valuationInvoiceService.RemoveRequestsFromInvoice(invoiceID,
req.ValuationRequestIDs)
if err != nil {
response.BadRequest(c, err.Error())
return
}

data := map[string]interface{}{
"invoice": valuationInvoice,
"requests_removed": len(req.ValuationRequestIDs),
"request_ids": req.ValuationRequestIDs,
```

```
"remaining_requests": len(valuationInvoice.ValuationRequests),
}

response.Success(c, "Requests removed from invoice successfully", data)
}

// GetAllValuationInvoices godoc
// @Summary Get all valuation invoices
// @Description Get paginated list of all valuation invoices
// @Tags valuation-invoices
// @Produce json
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Valuation invoices retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices [get]
func (h *ValuationInvoiceHandler) GetAllValuationInvoices(c *gin.Context) {
// Parse query parameters for pagination
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, total, err := h.valuationInvoiceService.GetAllValuationInvoices(limit,
offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve valuation invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"total": total,
```

```
"limit": limit,
"offset": offset,
}

response.Success(c, "Valuation invoices retrieved successfully", data)
}

// GetValuationInvoiceByID godoc
// @Summary Get valuation invoice by ID
// @Description Get specific valuation invoice by ID
// @Tags valuation-invoices
// @Produce json
// @Param id path string true "Invoice ID"
// @Success 200 {object} response.Response "Valuation invoice retrieved
successfully"
// @Failure 404 {object} response.Response "Valuation invoice not found"
// @Router /valuation-invoices/{id} [get]
func (h *ValuationInvoiceHandler) GetValuationInvoiceByID(c *gin.Context) {
invoiceID := c.Param("id")

valuationInvoice, err := h.valuationInvoiceService.GetValuationInvoiceByID(invoiceID)
if err != nil {
response.NotFound(c, "Valuation invoice not found")
return
}

// Include request count in response
data := map[string]interface{}{
"invoice": valuationInvoice,
"request_count": len(valuationInvoice.ValuationRequests),
}

response.Success(c, "Valuation invoice retrieved successfully", data)
}

// GetValuationInvoiceByInvoiceNumber godoc
// @Summary Get valuation invoice by invoice number
// @Description Get specific valuation invoice by invoice number
// @Tags valuation-invoices
// @Produce json
// @Param invoice_number path string true "Invoice number"
// @Success 200 {object} response.Response "Valuation invoice retrieved
successfully"
// @Failure 404 {object} response.Response "Valuation invoice not found"
```

```
// @Router /valuation-invoices/invoice-number/{invoice_number} [get]
func (h *ValuationInvoiceHandler) GetValuationInvoiceByInvoiceNumber(c
*gin.Context) {
invoiceNumber := c.Param("invoice_number")

valuationInvoice, err :=
h.valuationInvoiceService.GetValuationInvoiceByInvoiceNumber(invoiceNumber)
if err != nil {
response.NotFound(c, "Valuation invoice not found")
return
}

// Include request count in response
data := map[string]interface{}{
"invoice": valuationInvoice,
"request_count": len(valuationInvoice.ValuationRequests),
}

response.Success(c, "Valuation invoice retrieved successfully", data)
}

// UpdateValuationInvoice godoc
// @Summary Update valuation invoice
// @Description Update an existing valuation invoice
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param id path string true "Invoice ID"
// @Param request body services.UpdateValuationInvoiceRequest true "Update
valuation invoice request"
// @Success 200 {object} response.Response "Valuation invoice updated successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/{id} [put]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) UpdateValuationInvoice(c *gin.Context) {
invoiceID := c.Param("id")

var req services.UpdateValuationInvoiceRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}
```

```
if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err := h.valuationInvoiceService.UpdateValuationInvoice(invoiceID,
req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Valuation invoice updated successfully", valuationInvoice)
}

// DeleteValuationInvoice godoc
// @Summary Delete valuation invoice
// @Description Soft delete a valuation invoice
// @Tags valuation-invoices
// @Produce json
// @Param id path string true "Invoice ID"
// @Success 200 {object} response.Response "Valuation invoice deleted successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/{id} [delete]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) DeleteValuationInvoice(c *gin.Context) {
invoiceID := c.Param("id")

err := h.valuationInvoiceService.DeleteValuationInvoice(invoiceID)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Valuation invoice deleted successfully", nil)
}

// GetValuationInvoicesByStatus godoc
// @Summary Get valuation invoices by status
// @Description Get valuation invoices filtered by status
// @Tags valuation-invoices
// @Produce json
// @Param status path string true "Invoice status"
```

```
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Valuation invoices by status retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/status/{status} [get]
func (h *ValuationInvoiceHandler) GetValuationInvoicesByStatus(c *gin.Context) {
query := c.Param("status")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.SearchValuationInvoices(query,
limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to search valuation invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"query": query,
"limit": limit,
"offset": offset,
}

response.Success(c, "Invoice search completed successfully", data)
}

// GetPaidInvoices godoc
// @Summary Get paid invoices
```

```
// @Description Get paginated list of all paid valuation invoices
// @Tags valuation-invoices
// @Produce json
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Paid invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/paid [get]
func (h *ValuationInvoiceHandler) GetPaidInvoices(c *gin.Context) {
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetPaidInvoices(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve paid invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"status": "paid",
"limit": limit,
"offset": offset,
}

response.Success(c, "Paid invoices retrieved successfully", data)
}

// GetUnpaidInvoices godoc
// @Summary Get unpaid invoices
// @Description Get paginated list of all unpaid valuation invoices
```

```
// @Tags valuation-invoices
// @Produce json
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Unpaid invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/unpaid [get]
func (h *ValuationInvoiceHandler) GetUnpaidInvoices(c *gin.Context) {
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetUnpaidInvoices(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve unpaid invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"status": "unpaid",
"limit": limit,
"offset": offset,
}

response.Success(c, "Unpaid invoices retrieved successfully", data)
}

// GetInvoicesByDateRange godoc
// @Summary Get invoices by date range
// @Description Get valuation invoices within a specific date range
// @Tags valuation-invoices
```

```
// @Produce json
// @Param start_date query string true "Start date (YYYY-MM-DD format)"
// @Param end_date query string true "End date (YYYY-MM-DD format)"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Invoices by date range retrieved
successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/date-range [get]
func (h *ValuationInvoiceHandler) GetInvoicesByDateRange(c *gin.Context) {
startDate := c.Query("start_date")
endDate := c.Query("end_date")

if startDate == "" || endDate == "" {
response.BadRequest(c, "Both start_date and end_date are required")
return
}

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err :=
h.valuationInvoiceService.GetInvoicesByDateRange(startDate, endDate, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve invoices by date range")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
```

```
"start_date": startDate,
"end_date": endDate,
"limit": limit,
"offset": offset,
}

response.Success(c, "Invoices by date range retrieved successfully", data)
}

// GetInvoiceStatistics godoc
// @Summary Get invoice statistics
// @Description Get comprehensive invoice statistics
// @Tags valuation-invoices
// @Produce json
// @Success 200 {object} response.Response "Invoice statistics retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/statistics [get]
func (h *ValuationInvoiceHandler) GetInvoiceStatistics(c *gin.Context) {
statistics, err := h.valuationInvoiceService.GetInvoiceStatistics()
if err != nil {
response.InternalServerError(c, "Failed to retrieve invoice statistics")
return
}

response.Success(c, "Invoice statistics retrieved successfully", statistics)
}

// BulkUpdateInvoiceStatus godoc
// @Summary Bulk update invoice status
// @Description Update status for multiple invoices
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param request body map[string]interface{} true "Bulk status update request"
// @Success 200 {object} response.Response "Invoice statuses updated successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/bulk-status [patch]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) BulkUpdateInvoiceStatus(c *gin.Context) {
var req struct {
InvoiceIDs []string `json:"invoice_ids" validate:"required,min=1"`
Status string `json:"status" validate:"required"`
}
```

```
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

err := h.valuationInvoiceService.BulkUpdateInvoiceStatus(req.InvoiceIDs, req.Status)
if err != nil {
response.BadRequest(c, err.Error())
return
}

data := map[string]interface{}{
"invoice_ids": req.InvoiceIDs,
"status": req.Status,
"updated_count": len(req.InvoiceIDs),
}

response.Success(c, "Invoice statuses updated successfully", data)
}

// GetInvoicesByCustomer godoc
// @Summary Get invoices by customer
// @Description Get invoices for a specific customer through valuation requests
// @Tags valuation-invoices
// @Produce json
// @Param customer_id path string true "Customer ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Customer invoices retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/customer/{customer_id} [get]
func (h *ValuationInvoiceHandler) GetInvoicesByCustomer(c *gin.Context) {
customerID := c.Param("customer_id")

// Parse pagination parameters
limit := 10
offset := 0
```

```
if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err :=
h.valuationInvoiceService.GetInvoicesByCustomer(customerID, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve customer invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"customer_id": customerID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Customer invoices retrieved successfully", data)
}

// GetInvoicesByPartner godoc
// @Summary Get invoices by partner
// @Description Get invoices for a specific partner through valuation requests
// @Tags valuation-invoices
// @Produce json
// @Param partner_id path string true "Partner ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Partner invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/partner/{partner_id} [get]
func (h *ValuationInvoiceHandler) GetInvoicesByPartner(c *gin.Context) {
partnerID := c.Param("partner_id")
```

```
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetInvoicesByPartner(partnerID,
limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve partner invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"partner_id": partnerID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Partner invoices retrieved successfully", data)
}

// GetInvoicesByAgent godoc
// @Summary Get invoices by agent
// @Description Get invoices for a specific agent through valuation requests
// @Tags valuation-invoices
// @Produce json
// @Param agent_id path string true "Agent ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agent invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/agent/{agent_id} [get]
```

```
func (h *ValuationInvoiceHandler) GetInvoicesByAgent(c *gin.Context) {
agentID := c.Param("agent_id")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetInvoicesByAgent(agentID, limit,
offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agent invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"agent_id": agentID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agent invoices retrieved successfully", data)
}

// GetInvoicesByValuer godoc
// @Summary Get invoices by valuer
// @Description Get invoices for a specific valuer through valuation requests
// @Tags valuation-invoices
// @Produce json
// @Param valuer_id path string true "Valuer ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
```

```
// @Success 200 {object} response.Response "Valuer invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/valuer/{valuer_id} [get]
func (h *ValuationInvoiceHandler) GetInvoicesByValuer(c *gin.Context) {
valuerID := c.Param("valuer_id")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetInvoicesByValuer(valuerID,
limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve valuer invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"valuer_id": valuerID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Valuer invoices retrieved successfully", data)
}

// GetInvoicesWithMultipleRequests godoc
// @Summary Get invoices with multiple requests
// @Description Get invoices that have multiple valuation requests
// @Tags valuation-invoices
// @Produce json
```

```
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Multi-request invoices retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/multiple-requests [get]
func (h *ValuationInvoiceHandler) GetInvoicesWithMultipleRequests(c *gin.Context) {
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err :=
h.valuationInvoiceService.GetInvoicesWithMultipleRequests(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve multi-request invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"limit": limit,
"offset": offset,
}

response.Success(c, "Multi-request invoices retrieved successfully", data)
}

// GetInvoicesWithSingleRequest godoc
// @Summary Get invoices with single request
// @Description Get invoices that have only one valuation request
// @Tags valuation-invoices
// @Produce json
```

```
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Single-request invoices retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/single-request [get]
func (h *ValuationInvoiceHandler) GetInvoicesWithSingleRequest(c *gin.Context) {
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err :=
h.valuationInvoiceService.GetInvoicesWithSingleRequest(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve single-request invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"limit": limit,
"offset": offset,
}

response.Success(c, "Single-request invoices retrieved successfully", data)
}

// GetEmptyInvoices godoc
// @Summary Get empty invoices
// @Description Get invoices that have no valuation requests assigned
// @Tags valuation-invoices
// @Produce json
```

```
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Empty invoices retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /valuation-invoices/empty [get]
func (h *ValuationInvoiceHandler) GetEmptyInvoices(c *gin.Context) {
// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.GetEmptyInvoices(limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve empty invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"limit": limit,
"offset": offset,
}

response.Success(c, "Empty invoices retrieved successfully", data)
}

// UpdateInvoiceStatus godoc
// @Summary Update invoice status
// @Description Update the status of a valuation invoice
// @Tags valuation-invoices
// @Accept json
// @Produce json
// @Param id path string true "Invoice ID"
```

```
// @Param request body services.UpdateInvoiceStatusRequest true "Update invoice
status request"
// @Success 200 {object} response.Response "Invoice status updated successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /valuation-invoices/{id}/status [patch]
// @Security BearerAuth
func (h *ValuationInvoiceHandler) UpdateInvoiceStatus(c *gin.Context) {
invoiceID := c.Param("id")

var req services.UpdateInvoiceStatusRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

valuationInvoice, err := h.valuationInvoiceService.UpdateInvoiceStatus(invoiceID,
req.Status)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Invoice status updated successfully", valuationInvoice)
}

func (h *ValuationInvoiceHandler) SearchValuationInvoices(c *gin.Context) {
query := c.Query("q")
if query == "" {
response.BadRequest(c, "Search query is required")
return
}

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
```

```
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

valuationInvoices, err := h.valuationInvoiceService.SearchValuationInvoices(query,
limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to search valuation invoices")
return
}

data := map[string]interface{}{
"valuation_invoices": valuationInvoices,
"query": query,
"limit": limit,
"offset": offset,
}

response.Success(c, "Invoice search completed successfully", data)
}

// Helper function to parse integer from string
func parseInt0001(s string) (int, error) {
var result int
_, err := fmt.Sscanf(s, "%d", &result)
return result, err
}
```